

DocQuest

Complete Interview Preparation Guide

AI-Powered Document Question Answering System

Python · LangChain · FAISS · HuggingFace · Groq · Streamlit · OCR

Covers: Software Engineer · AI Engineer · ML Engineer · GenAI Engineer

Data Engineer · Python Developer · Campus Placements

18 Chapters · 150+ Interview Questions · Code Walkthroughs · Deep Concepts

Table of Contents

- Chapter 1:** Project Overview — Architecture, Data Flow, Execution
- Chapter 2:** Python Concepts — Imports, Functions, OOP, Decorators
- Chapter 3:** Streamlit — Execution Model, Session State, Caching, Deployment
- Chapter 4:** PDF Processing — PyPDF2, Internals, Encoding, Scanned PDFs
- Chapter 5:** DOCX Processing — XML Structure, docx2txt, python-docx
- Chapter 6:** OCR — Tesseract, Pipeline, Preprocessing, Confidence
- Chapter 7:** LangChain — LCEL, RAG Chain, All Core Components
- Chapter 8:** Text Splitting — Chunking, Overlap, Tradeoffs, Alternatives
- Chapter 9:** Embeddings — Vectors, Cosine Similarity, all-mpnet-base-v2
- Chapter 10:** FAISS — Index Types, from_texts(), similarity_search()
- Chapter 11:** LLMs — Transformers, Tokens, Groq, Llama 3.3 70B
- Chapter 12:** Q&A; Pipeline — Full Step-by-Step Walkthrough with Timing
- Chapter 13:** Environment Variables — .env, dotenv, Secrets Management
- Chapter 14:** Deployment — Git, Streamlit Cloud, Docker, Common Errors
- Chapter 15:** 150+ Interview Questions — Python, RAG, LLMs, System Design
- Chapter 16:** Project Viva — Ideal Answers, Wrong Answers, Follow-Ups
- Chapter 17:** Code Walkthrough — Line-by-Line with Explanations
- Chapter 18:** Optimization — Production Architecture, LangGraph, ChromaDB

Chapter 1: Project Overview

1.1 Problem Statement

The fundamental problem DocQuest solves: humans produce vast amounts of text in PDFs, Word documents, and images, but searching and extracting precise answers from them is slow, tedious, and error-prone. Traditional keyword search returns pages of text — it doesn't answer questions. DocQuest uses Retrieval-Augmented Generation (RAG) to let users ask natural language questions and receive precise, contextual answers grounded in the uploaded document.

Think of it this way: you have a 500-page legal contract. You want to know: 'What are the termination clauses?' A keyword search returns every page mentioning 'termination'. DocQuest reads and understands the document, retrieves the most relevant sections, and gives you a direct answer.

1.2 Why This Project Exists

Before Large Language Models (LLMs), search was keyword-based — computers matched strings, not meanings. The rise of transformer-based models (BERT, GPT) changed this: machines now understand semantic meaning. DocQuest bridges two worlds: (1) the ability to process arbitrary document formats (PDFs, DOCX, images), and (2) the ability of LLMs to reason about text. The project demonstrates the complete RAG pipeline — one of the most in-demand GenAI engineering patterns of 2024–2025.

1.3 Real-World Applications

- Legal: Query contracts, case files, regulatory documents
- Healthcare: Ask questions about medical records, clinical trial PDFs
- Finance: Summarise annual reports, extract figures from filings
- Education: Interactive textbook Q&A; for students
- HR: Screen resumes and answer policy handbook questions
- Research: Query scientific papers without reading them cover to cover
- Customer Support: Answer product manuals and FAQ PDFs automatically
- Government: Make public policy documents searchable and interactive

1.4 End-to-End Architecture

DocQuest follows a classic RAG (Retrieval-Augmented Generation) architecture. It has two phases: the Indexing Phase (document ingestion and embedding) and the Retrieval Phase (query processing and answer generation).

Layer	Technology	Purpose
UI Layer	Streamlit	File upload, question input, answer display
Document Ingestion	PyPDF2 / docx2txt / pytesseract	Extract raw text from any document type

Text Chunking	LangChain CharacterTextSplitter	Split text into manageable overlapping chunks
Embedding	HuggingFace all-mpnet-base-v2	Convert text chunks to 768-dim vectors
Vector Store	FAISS	Store and index all chunk vectors
Retrieval	FAISS similarity_search	Find top-k most relevant chunks for a query
Generation	Groq Llama 3.3 70B via LangChain	Generate the final natural language answer
Secrets	python-dotenv / Streamlit Secrets	Manage API keys securely

1.5 Complete Execution Flow

Below is the step-by-step sequence of what happens from the moment a user opens DocQuest to when they see an answer:

Step 1 — App Launch

Streamlit starts, loads environment variables (.env), initialises session state. No computation happens yet.

Step 2 — Document Upload

User uploads a PDF, DOCX, or image. Streamlit stores the bytes in `st.session_state`. The file is NOT yet processed.

Step 3 — Text Extraction

On pressing Process, the app detects the file type (extension check), calls the appropriate extractor: PyPDF2 for PDFs, docx2txt for DOCX, pytesseract for images. Raw text string is produced.

Step 4 — Text Chunking

LangChain's CharacterTextSplitter splits the raw text into chunks (e.g., 1000 chars each, 200 char overlap). Result: a Python list of strings, each being one chunk.

Step 5 — Embedding

HuggingFaceEmbeddings loads the all-mpnet-base-v2 model. Each chunk string is passed through the model, producing a 768-dimensional float vector. All vectors are collected.

Step 6 — FAISS Indexing

`FAISS.from_texts()` builds a flat index. It stores all vectors and maps each vector back to its source chunk text. The index lives in RAM.

Step 7 — User Asks a Question

User types a question in the Streamlit text input. On pressing Ask, the question string is captured.

Step 8 — Query Embedding

The same HuggingFaceEmbeddings model embeds the user's question into a 768-dim vector.

Step 9 — Similarity Search

FAISS compares the query vector against all stored chunk vectors using cosine similarity (or L2). Returns top-k (e.g., 4) most similar chunks.

Step 10 — Prompt Construction

LangChain builds a prompt: it combines the retrieved chunks (context) with the user's question using a `PromptTemplate`.

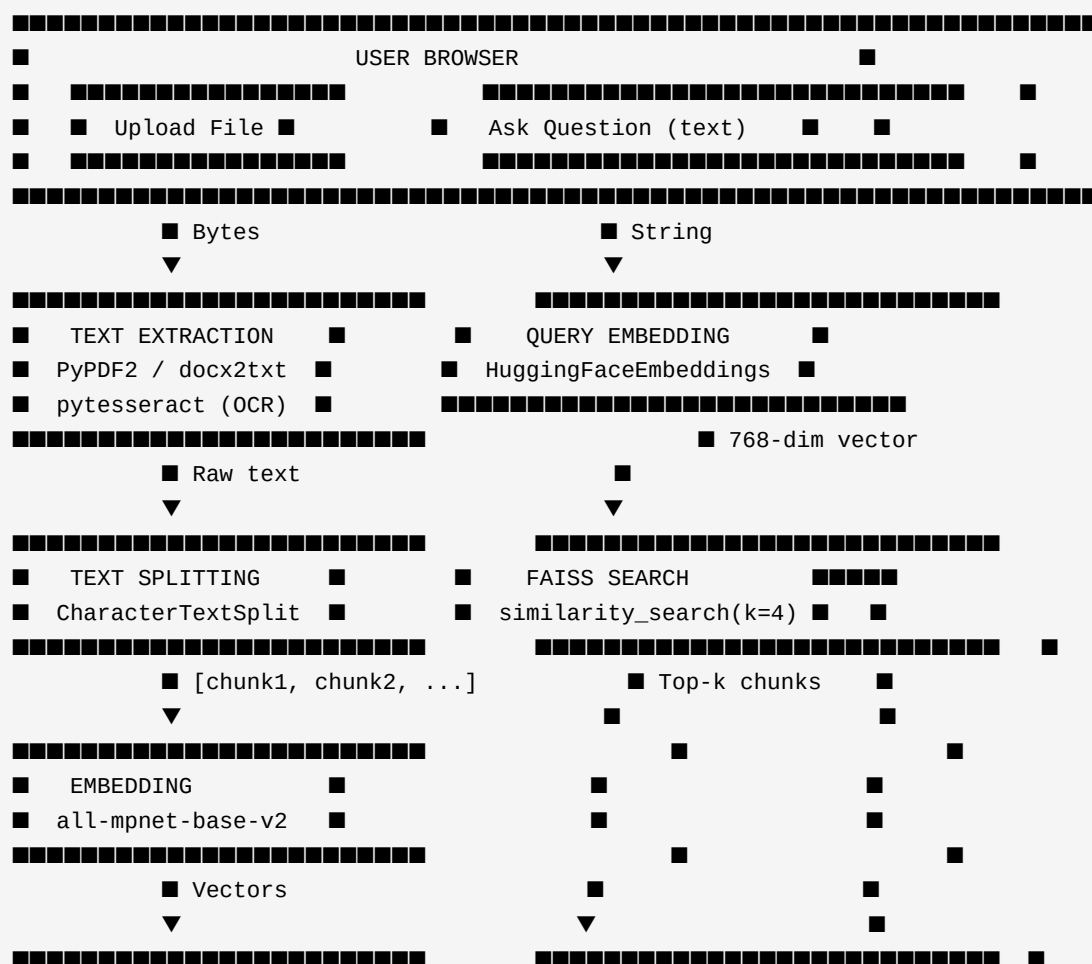
Step 11 — LLM Call

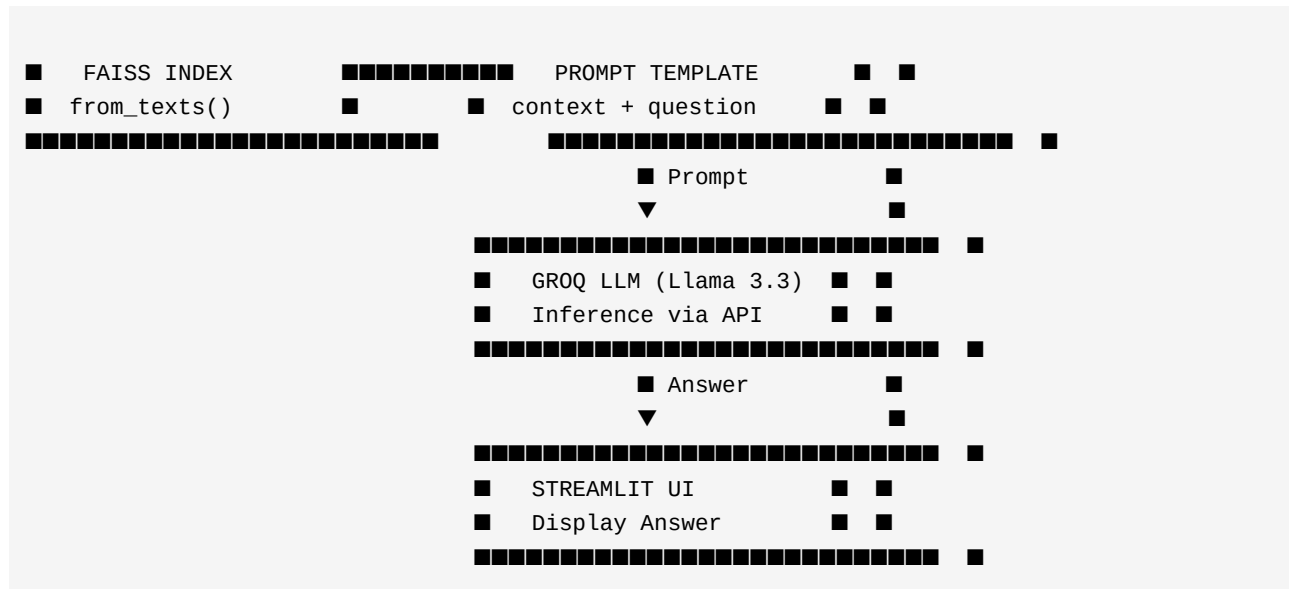
The prompt is sent to the Groq API (Llama 3.3 70B). Groq's inference hardware processes the tokens and streams back the answer.

Step 12 — Answer Display

Streamlit renders the LLM's answer in the UI. The session state stores conversation history for multi-turn capability.

1.6 Data Flow Diagram (ASCII)





1.7 Module Interaction Summary

Every module in DocQuest has a clear responsibility. They interact through well-defined Python objects: strings, lists of strings, lists of Document objects, and vector stores.

Module	Input	Output	Interacts With
app.py (Streamlit)	User actions	UI renders	All modules
PyPDF2 / docx2txt / pytesera	File bytes	Raw text string	app.py
CharacterTextSplitter	Raw text	List[str] chunks	FAISS module
HuggingFaceEmbeddings	List[str]	List[float[768]]	FAISS module
FAISS	Vectors + chunks	VectorStore object	Retriever chain
PromptTemplate	Context + question	Formatted prompt	LLM chain
ChatGroq / ChatOpenAI	Prompt string	Answer string	app.py
session_state	Any Python object	Persists across reruns	app.py

Chapter 2: Python Concepts Used in DocQuest

2.1 Imports and Modules

Python's import system is one of the most fundamental concepts. When you write 'import os', Python searches sys.path for a file called os.py (or a package folder called os/). A **module** is any .py file. A **package** is a directory containing an __init__.py file. A **library** is a collection of related modules.

```
import os                    # Standard library module
import streamlit as st      # Third-party library, aliased as 'st'
from langchain.text_splitter import CharacterTextSplitter
# ↑ imports a specific class from a submodule
from dotenv import load_dotenv # imports a function
from PyPDF2 import PdfReader  # imports a class
```

Why alias? 'import streamlit as st' means everywhere you'd write 'streamlit.something', you write 'st.something'. It saves typing and improves readability. This is a Python convention for popular libraries.

■ **Tip:** In interviews, be able to explain the difference between 'import X', 'from X import Y', and 'from X import Y as Z'. Also explain why circular imports are a problem.

2.2 Virtual Environments

A virtual environment is an isolated Python installation for your project. Without it, all projects share the same Python packages, causing version conflicts. If Project A needs langchain==0.1 and Project B needs langchain==0.2, they will break each other.

```
# Create virtual environment
python -m venv venv

# Activate (Linux/Mac)
source venv/bin/activate

# Activate (Windows)
venv\Scripts\activate

# Install packages INTO the virtual environment
pip install streamlit langchain faiss-cpu

# Save dependencies
pip freeze > requirements.txt

# Recreate on another machine
pip install -r requirements.txt
```

■ **Tip:** venv creates a copy of the Python interpreter and pip. When activated, 'python' and 'pip' point to the venv copies, not the system ones. Packages installed go into venv/lib/python3.x/site-packages/

2.3 Functions

Functions are reusable blocks of code. In DocQuest, every major operation is wrapped in a function: `extract_text_from_pdf()`, `create_vector_store()`, `get_answer()`. This follows the Single Responsibility Principle.

```
def extract_text_from_pdf(pdf_file) -> str:
    """Extract all text from an uploaded PDF file.

    Args:
        pdf_file: A file-like object (from st.file_uploader)

    Returns:
        str: Concatenated text from all pages
    """
    reader = PdfReader(pdf_file)
    text = ""
    for page in reader.pages:
        text += page.extract_text()
    return text
```

Key Python function features used in DocQuest: default arguments, return type annotations (`-> str`), docstrings, and returning early on error conditions.

2.4 Dictionaries, Lists, Tuples

```
# List – ordered, mutable, used for chunks
chunks = ["chunk1 text", "chunk2 text", "chunk3 text"]
chunks.append("new chunk")    # add to end
chunks[0]                     # access by index

# Dictionary – key-value, used for session_state, metadata
session_state = {"chat_history": [], "vectorstore": None}
session_state["chat_history"].append({"role": "user", "content": "Hello"})

# Tuple – ordered, IMMUTABLE, used for fixed configs
PAGE_SIZE = (8.5, 11) # width, height in inches
SUPPORTED_FORMATS = ("pdf", "docx", "jpg", "png")
```

■ **Tip:** LangChain Document objects have two fields: `page_content` (str) and `metadata` (dict). The metadata dict stores source file name, page number, etc.

2.5 Session State

Streamlit's `st.session_state` is a special dictionary that persists across script reruns. Since Streamlit re-executes the entire script on every user interaction, normal Python variables would be reset. `session_state` solves this.

```
# Initialise with a default value (only on first run)
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []

if "vectorstore" not in st.session_state:
    st.session_state.vectorstore = None
```

```
# Use it anywhere in the script
st.session_state.chat_history.append(message)
# This persists even when Streamlit reruns the script
```

Internally, `session_state` is stored server-side, keyed by the user's browser session. Each user gets their own isolated `session_state`.

2.6 Environment Variables

```
# .env file (NEVER commit to Git)
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXX
HUGGINGFACE_TOKEN=hf_XXXXXXXXXXXXX

# In Python:
from dotenv import load_dotenv
import os

load_dotenv() # reads .env, sets os.environ entries
api_key = os.getenv("GROQ_API_KEY") # returns None if missing
api_key = os.environ["GROQ_API_KEY"] # raises KeyError if missing

# os.getenv() is safer – use it when the key is optional
# os.environ[] is stricter – use it when the key is required
```

■ ■ **Never hardcode API keys in source code. Git history is permanent — even if you delete the key later, it remains visible in commits. Use `.env` + `.gitignore`.**

2.7 File Handling

Streamlit's `file_uploader` returns a `UploadedFile` object — a file-like object (it has `.read()`, `.seek()`, `.name` attributes). Most document processing libraries accept file-like objects directly.

```
# File-like objects
uploaded_file = st.file_uploader("Upload PDF")
if uploaded_file:
    # Read as bytes
    bytes_data = uploaded_file.read()

    # Reset position (important!)
    uploaded_file.seek(0)

    # Pass to library
    reader = PdfReader(uploaded_file)

    # Get filename and extension
    filename = uploaded_file.name          # "report.pdf"
    extension = filename.split(".")[-1]    # "pdf"

# Context manager for real files
with open("output.txt", "w") as f:
```

```
f.write("extracted text")
# File is automatically closed here
```

2.8 Exception Handling

```
# Basic pattern
try:
    text = extract_text_from_pdf(uploaded_file)
except Exception as e:
    st.error(f"Failed to extract text: {e}")
    text = ""

# Specific exceptions (more Pythonic)
try:
    reader = PdfReader(uploaded_file)
except PdfReadError as e:
    st.error("Corrupted PDF file")
except FileNotFoundError:
    st.error("File not found")
except Exception as e: # catch-all at the end
    st.error(f"Unexpected error: {e}")
finally:
    # Always runs – good for cleanup
    uploaded_file.seek(0)
```

■ **Tip:** In interviews: explain the difference between 'except Exception' (catches most errors) and bare 'except' (catches everything including SystemExit, KeyboardInterrupt — almost never do this).

2.9 Object-Oriented Concepts Used By Libraries

Even if you didn't write classes yourself, every library object is a class instance. Understanding OOP helps you use them effectively:

```
# PdfReader is a class. PdfReader(file) creates an INSTANCE.
reader = PdfReader(pdf_file)
# reader.pages is an ATTRIBUTE (list of Page objects)
# reader.metadata is an ATTRIBUTE (dict-like object)

# HuggingFaceEmbeddings is a class
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2"
)
# embeddings.embed_query() is a METHOD
vector = embeddings.embed_query("What is AI?")

# FAISS is a class with CLASS METHODS (no instance needed)
vectorstore = FAISS.from_texts(chunks, embeddings)
# from_texts is a @classmethod – it creates and returns an instance
```

Key OOP concepts in use: classes, instances, attributes, methods, class methods, inheritance (ChatGroq inherits from BaseLLM), and duck typing (any file-like object works with PdfReader).

2.10 Context Managers

```
# Context managers use __enter__ and __exit__
with open("file.txt", "r") as f:
    content = f.read()
# f is automatically closed even if an exception occurs

# pdfplumber uses context managers
with pdfplumber.open("doc.pdf") as pdf:
    for page in pdf.pages:
        text = page.extract_text()

# Writing your own context manager
from contextlib import contextmanager

@contextmanager
def timer(operation_name):
    import time
    start = time.time()
    yield
    elapsed = time.time() - start
    print(f"{operation_name} took {elapsed:.2f}s")

with timer("Embedding"):
    vectors = embeddings.embed_documents(chunks)
```

Chapter 3: Streamlit — Complete Guide

3.1 What is Streamlit and Why Use It?

Streamlit is a Python framework for building web applications without writing HTML, CSS, or JavaScript. You write a Python script, and Streamlit converts it into an interactive web app. It was purpose-built for data scientists and ML engineers who need to share models and pipelines quickly.

Alternative: Flask/Django require separate frontend code (HTML templates, JavaScript). Streamlit collapses everything into one Python file. The tradeoff: less flexibility for complex UIs, but 10x faster to build for data-centric apps.

3.2 Streamlit's Execution Model — The Most Important Concept

THIS IS THE MOST COMMON STREAMLIT INTERVIEW TOPIC. Understand it deeply.

Streamlit re-executes your entire Python script from top to bottom on every user interaction. Every widget click, text input, file upload, button press — each triggers a complete script re-run.

```
# This entire file runs EVERY TIME the user interacts

import streamlit as st

# Line 1: runs on every rerun
name = st.text_input("Your name") # widget

# Line 2: runs on every rerun
if name:
    st.write(f"Hello {name}!") # display

# If user types "Alice", the ENTIRE script runs again.
# name will equal "Alice" this time.
```

Why does Streamlit do this? It makes the programming model simple. You think imperatively: 'if this widget has value X, show Y'. No callbacks, no event listeners, no state machines. The tradeoff is performance: complex computation reruns unnecessarily.

Solution: `@st.cache_data` and `@st.cache_resource` decorators. They memoize function results so they don't recompute on reruns.

3.3 Caching

```
@st.cache_data
def load_and_process_document(uploaded_file_bytes, filename):
    """Cache document processing results."""
    # This function runs only when inputs change
    # On reruns with same inputs: cached result returned instantly
    text = extract_text(uploaded_file_bytes, filename)
```

```

        chunks = split_text(text)
        return chunks

@st.cache_resource
def get_embedding_model():
    """Cache the embedding model – loaded once, reused."""
    # @cache_resource for objects that can't be pickled or are large
    # HuggingFace models are perfect candidates
    return HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-mpnet-base-v2"
    )

```

@st.cache_data: caches data (returns serialisable objects like DataFrames, lists, strings). Creates a copy for each caller. Safe for data.

@st.cache_resource: caches resources (returns single shared instance — ML models, database connections). Does NOT copy. Faster but not thread-safe for mutation.

3.4 Session State

```

# Pattern 1: initialise with defaults
if "messages" not in st.session_state:
    st.session_state.messages = []

# Pattern 2: use get() with default
history = st.session_state.get("history", [])

# Pattern 3: dot notation vs dict notation (both work)
st.session_state.vectorstore = vectorstore # dot
st.session_state["vectorstore"] = vectorstore # dict

# Common mistake: modifying a list in session_state
# WRONG (doesn't trigger rerun):
# temp = st.session_state.messages
# temp.append(msg) # modifying local copy
# RIGHT:
st.session_state.messages.append(msg) # direct mutation

```

3.5 Key Widgets Used in DocQuest

```

# File uploader
uploaded = st.file_uploader(
    label="Upload Document",
    type=["pdf", "docx", "jpg", "png"],
    accept_multiple_files=False
)

# Text input (single line)
question = st.text_input("Ask a question:", key="question_input")

```

```

# Text area (multiline)
question = st.text_area("Ask a question:", height=100)

# Button
if st.button("Process Document", type="primary"):
    # runs when button clicked
    process_document(uploaded)

# Spinner (loading indicator)
with st.spinner("Processing document..."):
    result = expensive_operation()

# Success/Error/Warning/Info boxes
st.success("Document processed!")
st.error("Failed to extract text.")
st.warning("No text found in document.")
st.info("Upload a document to begin.")

# Sidebar
with st.sidebar:
    st.title("Settings")
    k = st.slider("Number of results", 1, 10, 4)

```

3.6 Displaying Conversation History

```

# Chat messages (Streamlit 1.25+)
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# Older pattern with columns
for i, msg in enumerate(st.session_state.chat_history):
    if msg["role"] == "user":
        st.write(f"**You:** {msg['content']}")
    else:
        st.write(f"**DocQuest:** {msg['content']}")

```

3.7 Streamlit Deployment

Streamlit Cloud (share.streamlit.io) is the easiest free deployment. Requirements:

- GitHub repository (public or private with access granted)
- requirements.txt listing all pip dependencies
- packages.txt for system-level packages (e.g., tesseract-ocr, poppler-utils)
- Secrets management: add GROQ_API_KEY in Streamlit Cloud dashboard → app settings → secrets
- App entrypoint: usually app.py or streamlit_app.py

```
# .streamlit/secrets.toml (local development)
GROQ_API_KEY = "gsk_xxxx"

# In your Python code:
import streamlit as st
api_key = st.secrets["GROQ_API_KEY"]

# This works both locally (from secrets.toml)
# and in Streamlit Cloud (from dashboard secrets)
```

3.8 Interview Questions — Streamlit

Q: Why does Streamlit rerun the script on every interaction?

A: Streamlit's design philosophy is reactive programming: the UI is a pure function of the current state. On any interaction, the state changes, so the function (script) must re-execute to recompute the UI. This makes programming simple (no callbacks) at the cost of performance, which caching mitigates.

Q: What is the difference between `@st.cache_data` and `@st.cache_resource`?

A: `@st.cache_data` is for data (pickles and copies results — safe for concurrent users).
`@st.cache_resource` is for global resources like ML models (shared instance, no copy — faster but callers must not mutate it). Use `@st.cache_resource` for HuggingFace models.

Q: How do you persist state between Streamlit reruns?

A: Using `st.session_state` — a dictionary-like object that persists for the duration of a user session. Each user gets their own isolated `session_state`. Keys can be set with dot notation or bracket notation.

Q: What is a Streamlit key and why is it important?

A: A key is a unique identifier for a widget. It prevents Streamlit from confusing two widgets with the same label. It also allows you to control a widget's value via `session_state[key]`. Without keys, widgets inside loops or conditionals may behave unexpectedly.

Chapter 4: PDF Processing

4.1 What Is a PDF Internally?

PDF (Portable Document Format) is NOT a container of text like a .txt file. It is a page description language — a sequence of drawing instructions. A PDF tells the renderer: 'draw glyph A at position (x=102, y=740), draw glyph t at (x=108, y=740)...' It doesn't store words or sentences as strings. This is why PDF text extraction is harder than reading a .txt file.

PDF internals: a PDF file contains objects (numbers, strings, arrays, dictionaries, streams). A page is a dictionary pointing to a content stream (the drawing instructions) and a resources dictionary (fonts, images). Text in a PDF is encoded as sequences of glyph IDs, which are mapped back to Unicode characters through a ToUnicode table — which may or may not be present.

4.2 How PyPDF2 Works

```
from PyPDF2 import PdfReader

reader = PdfReader("document.pdf")

# PdfReader parses the PDF cross-reference table
# to find all objects

print(len(reader.pages)) # number of pages

# Extract text from page 0
text = reader.pages[0].extract_text()

# What extract_text() does internally:
# 1. Finds the page's content stream
# 2. Parses PDF drawing operators (Tf, Tj, TJ, etc.)
# 3. Maps glyph IDs to Unicode via ToUnicode CMap
# 4. Assembles characters in order
# 5. Returns a string (may have spacing issues)
```

Limitations of PyPDF2's `extract_text()`: (1) Text may appear in wrong order — PDFs store text in drawing order, not reading order. (2) Spaces between words may be lost. (3) Two-column layouts break. (4) Scanned PDFs return empty string (no text layer exists).

4.3 OCR vs Text Extraction

Aspect	Text Extraction (PyPDF2)	OCR (pytesseract)
Works on	Searchable/digital PDFs	Scanned PDFs, images, photos of docs
Speed	Very fast (milliseconds/page)	Slow (seconds/page)
Accuracy	Perfect (if encoding correct)	95-99% (errors on bad scans)

Requires	Text layer in PDF	Tesseract installed, image conversion
Output	Preserves logical order	May struggle with complex layouts
Tables	Loses structure	Also loses structure (use specialized OCR)

4.4 Scanned vs Searchable PDFs

A **searchable PDF** has a text layer — you can copy-paste text from it. PyPDF2 can extract this text directly. A **scanned PDF** is essentially a collection of images of pages. There is no text layer. PyPDF2 returns an empty string or garbage. You must convert each page to an image and run OCR.

```
# Detecting if a PDF is scanned
reader = PdfReader("document.pdf")
text = reader.pages[0].extract_text()
if not text.strip():
    # Likely scanned – use OCR pipeline
    from pdf2image import convert_from_bytes
    images = convert_from_bytes(pdf_bytes)
    text = pytesseract.image_to_string(images[0])
```

4.5 Interview Questions — PDF

Q: Why does PyPDF2 sometimes return garbled or missing text?

A: PDFs store text as glyph IDs with font encoding, not as Unicode. If the PDF's ToUnicode CMap is missing or incorrect (common in older PDFs and some form generators), PyPDF2 cannot map glyphs back to characters. Also, PDFs may use custom font encodings. In such cases, use pdfplumber (better text extraction) or OCR.

Q: What is the difference between a scanned and searchable PDF?

A: A searchable PDF has an embedded text layer (actual Unicode characters) that can be selected and searched. A scanned PDF is an image embedded in a PDF container — it has no text layer. Searchable PDFs work with PyPDF2.extract_text(). Scanned PDFs require OCR: convert pages to images, then run Tesseract.

Q: How would you handle a PDF where text extraction returns empty string?

A: First, try pdfplumber which has more robust extraction. If still empty, the PDF is likely scanned. Convert pages to images using pdf2image.convert_from_bytes() and apply pytesseract.image_to_string(). Optionally preprocess the image (grayscale, denoise, skew) to improve OCR accuracy.

Chapter 5: DOCX Processing

5.1 DOCX Internal Structure

A .docx file is a ZIP archive. Unzip it and you find XML files. The main content is in word/document.xml. Paragraphs are elements; runs (text spans with consistent formatting) are elements; text is in elements. Images, headers, footers are in separate XML files.

```
# Manually inspect a docx file
import zipfile
with zipfile.ZipFile("document.docx", "r") as z:
    print(z.namelist())
    # ['[Content_Types].xml', '_rels/.rels',
    #  'word/document.xml', 'word/styles.xml', ...]

    with z.open("word/document.xml") as f:
        content = f.read().decode("utf-8")
        print(content[:500]) # raw XML
```

5.2 docx2txt vs python-docx

```
# docx2txt – simple, fast, extracts all text
import docx2txt
text = docx2txt.process("document.docx")
# Returns one big string. Fast, minimal dependencies.

# python-docx – full control, preserves structure
from docx import Document
doc = Document("document.docx")

# Iterate paragraphs
for para in doc.paragraphs:
    if para.style.name.startswith("Heading"):
        print(f"[HEADING] {para.text}")
    else:
        print(para.text)

# Access tables
for table in doc.tables:
    for row in table.rows:
        for cell in row.cells:
            print(cell.text, end=" | ")
```

DocQuest likely uses docx2txt for simplicity. Use python-docx when you need to: preserve document structure, extract tables, modify the document, or read metadata like author, date.

Chapter 6: OCR — Optical Character Recognition

6.1 What is OCR?

OCR is the process of converting images of text into machine-readable text. A camera photograph of a page, a scanned document, or a screenshot — these are images. OCR algorithms analyze pixel patterns to recognize letters and words.

OCR Pipeline: Image Input → Preprocessing (grayscale, denoise, deskew, binarize) → Layout Analysis (find text regions, columns) → Line Detection → Character Segmentation → Character Recognition → Post-processing (spell check, dictionary lookup) → Output Text.

6.2 Tesseract and pytesseract

```
import pytesseract
from PIL import Image
import cv2
import numpy as np

# Basic usage
img = Image.open("scanned_page.png")
text = pytesseract.image_to_string(img)

# Specify language
text = pytesseract.image_to_string(img, lang="eng")

# Get confidence scores
data = pytesseract.image_to_data(img, output_type=pytesseract.Output.DICT)
# data["text"] = list of recognized words
# data["conf"] = list of confidence scores (0-100, -1 = non-word)

# Image preprocessing for better results
def preprocess(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)          # grayscale
    gray = cv2.threshold(gray, 0, 255,                    # binarize
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1]
    return gray
```

Tesseract is an open-source OCR engine originally developed by HP, now maintained by Google. It uses LSTM (Long Short-Term Memory) neural networks for character recognition. pytesseract is the Python wrapper — it calls the Tesseract binary installed on your system.

6.3 Why OCR Makes Mistakes

- Low resolution: text below 300 DPI is hard to segment accurately
- Poor contrast: light text on light background, faded ink
- Skewed pages: even 2-degree tilt confuses line detection
- Handwriting: printed text models don't generalise to handwriting
- Complex fonts: decorative, cursive, or unusual fonts reduce accuracy
- Mixed languages: Tesseract must be told which language(s) to expect
- Noise and stains: random pixels that resemble characters
- Tables and complex layouts: OCR reads across columns, mixing content

6.4 Interview Questions — OCR

Q: When would you use OCR versus text extraction in DocQuest?

A: Use text extraction (PyPDF2) first — it's fast and accurate for digital PDFs. Fall back to OCR when: (1) text extraction returns empty or near-empty string, (2) the file is an image (JPEG, PNG), or (3) the file is a scanned PDF. A simple heuristic: if extracted text length < 100 characters for a multi-page doc, assume scanned.

Q: How can you improve OCR accuracy for poor-quality scans?

A: Preprocessing pipeline: (1) Grayscale conversion removes colour noise, (2) Gaussian blur reduces salt-and-pepper noise, (3) Otsu thresholding binarizes optimally, (4) Deskewing corrects page tilt using Hough transform, (5) Morphological operations remove stray marks. Also try increasing DPI (convert_from_path with dpi=300+) and using Tesseract's page segmentation modes (--psm 6 for single block of text).

Chapter 7: LangChain — Complete Guide

7.1 What is LangChain and Why Does It Exist?

LangChain is a framework for building applications that use Large Language Models (LLMs). Before LangChain, every developer had to manually: (1) manage prompt construction, (2) parse LLM outputs, (3) implement retrieval pipelines, (4) handle conversation history, (5) orchestrate multi-step reasoning. LangChain standardises all of this into composable building blocks.

Real-world analogy: LangChain is to LLM development what Django is to web development. Django gives you models, views, URL routing, ORM — so you don't reinvent the wheel. LangChain gives you prompt templates, chains, retrievers, memory — so you don't reinvent the RAG pipeline.

7.2 Core Components

Prompt Templates: Standardise prompt construction. Define placeholders (variables) that get filled with runtime values. Ensures consistent, structured prompts.

LLMs / Chat Models: Wrappers around LLM APIs (OpenAI, Groq, Anthropic, Cohere). Standardise the call interface so you can swap models without changing pipeline code.

Output Parsers: Parse LLM text output into structured Python objects (lists, JSON, Pydantic models). LLM always returns text; parsers extract the useful structure.

Document Loaders: Load documents from various sources (PDFs, URLs, databases, S3) and return a standardised list of Document objects.

Text Splitters: Split large documents into smaller chunks. Multiple strategies: by character, by token, by sentence, by markdown heading, recursively.

Embeddings: Wrappers around embedding models. Call `.embed_documents()` or `.embed_query()` — same interface regardless of underlying model.

Vector Stores: Wrappers around vector databases (FAISS, Chroma, Pinecone). Store and retrieve embeddings. Call `.similarity_search()` for nearest-neighbour lookup.

Retrievers: Abstract retrieval interface. A retriever takes a query and returns relevant Documents. FAISS VectorStore can be converted to a retriever: `vectorstore.as_retriever()`.

Chains: Sequence of operations. Deprecated approach: `LLMChain(prompt, llm)`. Modern approach: LCEL (LangChain Expression Language) pipes: `prompt | llm | parser`.

Memory: Maintains conversation history across turns. Types: `ConversationBufferMemory` (full history), `ConversationSummaryMemory` (compressed), `ConversationWindowMemory` (last N turns).

Agents: LLM-driven decision makers. The LLM decides which tool to call and in what order. Useful for open-ended tasks requiring reasoning and multiple tool calls.

Tools: Functions the agent can call: web search, calculator, code interpreter, database queries. Defined with name, description, and function.

7.3 The Modern LangChain Architecture (LCEL)

LangChain Expression Language (LCEL) replaces the old chain classes. It uses Python's | (pipe) operator to compose components. Each component is a Runnable — it has .invoke(), .stream(), .batch(), and .ainvoke() methods.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_groq import ChatGroq
from langchain_core.output_parsers import StrOutputParser

# Define components
prompt = ChatPromptTemplate.from_template(
    "Answer the question based on the context below.\n"
    "Context: {context}\n"
    "Question: {question}\n"
    "Answer:"
)
llm = ChatGroq(model="llama-3.3-70b-versatile", temperature=0)
parser = StrOutputParser()

# Compose using LCEL pipe operator
chain = prompt | llm | parser

# Invoke
answer = chain.invoke({
    "context": "Python was created by Guido van Rossum in 1991.",
    "question": "Who created Python?"
})
# answer = "Python was created by Guido van Rossum."

# Stream tokens
for chunk in chain.stream({"context": "...", "question": "..."}):
    print(chunk, end="", flush=True)
```

7.4 RAG Chain in DocQuest

```
from langchain.chains import RetrievalQA
from langchain_core.prompts import PromptTemplate

# Create a retriever from FAISS vectorstore
retriever = vectorstore.as_retriever(
    search_type="similarity", # or "mmr" for diverse results
    search_kwargs={"k": 4}    # return top 4 chunks
)

# Define the prompt
```

```

prompt = PromptTemplate(
    template="""You are a helpful assistant. Answer the question
based ONLY on the provided context. If the answer is not in
the context, say "I don't know based on the provided document."

Context:
{context}

Question: {question}

Answer: """,
    input_variables=["context", "question"]
)

# Build the RAG chain (older API)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",    # "stuff" = concatenate all chunks into prompt
    retriever=retriever,
    chain_type_kwargs={"prompt": prompt}
)

# Invoke
result = qa_chain({"query": "What are the key findings?"})
print(result["result"])

```

Chain types: 'stuff' (concatenate all retrieved chunks into one prompt — simple, good for small contexts), 'map_reduce' (process each chunk separately, then combine), 'refine' (iteratively refine answer with each chunk), 'map_rerank' (score each chunk's answer, return best).

7.5 Interview Questions — LangChain

Q: What is LCEL and why was it introduced?

A: LCEL (LangChain Expression Language) is the modern way to compose LangChain components using the | pipe operator. It was introduced to replace the old chain classes (LLMChain, RetrievalQA) which were verbose and inconsistent. LCEL components are all Runnables with a standard interface (.invoke, .stream, .batch, .ainvoke), making async, streaming, and parallel execution trivial.

Q: What is the difference between a Chain and an Agent in LangChain?

A: A Chain follows a predetermined, fixed sequence of steps. You define the flow at construction time. An Agent uses an LLM to dynamically decide what to do next — which tool to call, whether to call another tool, or when to return the final answer. Chains are predictable and fast; Agents are flexible but slower and can get stuck in loops.

Q: What is RAG and why is it needed?

A: RAG (Retrieval-Augmented Generation) extends an LLM's knowledge beyond its training data by retrieving relevant documents at inference time and providing them as context. Without RAG, an LLM can only answer from its training data (which has a knowledge cutoff and no private data). With RAG, the LLM can answer questions about any document you provide, reducing hallucination since answers are grounded in real text.

Chapter 8: Text Splitting

8.1 Why Chunking is Required

LLMs have a context window limit (measured in tokens). Llama 3.3 70B has a 128K token context window, but: (1) sending an entire 500-page book exceeds any limit, (2) LLMs pay attention to ALL tokens in the context — more tokens = slower and more expensive inference, (3) embedding models have much smaller limits (all-mpnet-base-v2: 384 tokens). Chunking solves this by splitting documents into manageable pieces.

The second reason: precision. If you have a 1000-page book, the relevant answer to 'What is the capital of France?' is in one sentence. Sending all 1000 pages wastes context window and dilutes the relevant signal. Chunking + retrieval sends only the 4 most relevant chunks — focused, precise, cheap.

8.2 CharacterTextSplitter

```
from langchain.text_splitter import CharacterTextSplitter

splitter = CharacterTextSplitter(
    separator="\n",    # split on newlines first
    chunk_size=1000,   # target: 1000 characters per chunk
    chunk_overlap=200, # each chunk shares 200 chars with the previous
    length_function=len # use len() to measure chunk size
)

chunks = splitter.split_text(raw_text)
# Returns: List[str]

# Or split Document objects
docs = splitter.split_documents(documents)
# Returns: List[Document] – preserves metadata
```

How it works: (1) split text on separator (newlines), (2) greedily merge splits until chunk_size is reached, (3) when limit exceeded, start a new chunk but include the last chunk_overlap characters from the previous chunk.

8.3 Chunk Overlap — Why It Matters

Imagine the answer to a question spans the boundary between two chunks. Without overlap, the relevant context is split in half: chunk 1 has the first half, chunk 2 has the second half. When FAISS retrieves the top chunk, it only gets half the answer.

With overlap (e.g., 200 characters), each chunk shares its last 200 chars with the next chunk's first 200 chars. The critical boundary text appears in BOTH chunks. When FAISS retrieves either chunk, the answer is complete.

```
# Example with chunk_size=50, chunk_overlap=10
text = "ABCDEFGH IJKLMNOPQRSTUVWXYZ" * 4 # 104 chars

# Without overlap:
```

```
# Chunk 1: ABCDE...Z (0-49)
# Chunk 2: ABCDE...Z (50-99)  <- no connection to chunk 1

# With overlap of 10:
# Chunk 1: ABCDE...Z (0-49)
# Chunk 2: ...last10ofChunk1...newContent (40-89)  <- overlap!
# Chunk 3: ...last10ofChunk2...newContent (80-103)
```

8.4 Chunk Size Tradeoffs

Chunk Size	Pros	Cons	Best For
Small (200-400 chars)	High precision retrieval, cheap embeddings	Contexts may be too narrow, splits sentences in middle of thoughts	FAQs, fast and simple queries
Medium (800-1200 chars)	Balances context and precision	Some irrelevant context included	General documents (default choice)
Large (2000-4000 chars)	Rich context for complex questions	More noise, embedding model may equalize	Legal or scientific documents with long arguments

8.5 Alternative Splitters

```
# RecursiveCharacterTextSplitter – recommended default
# Tries to split on ["\n\n", "\n", " ", ""] in order
# Better preserves paragraph/sentence boundaries
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)

# TokenTextSplitter – splits by token count (more accurate for LLMs)
from langchain.text_splitter import TokenTextSplitter
splitter = TokenTextSplitter(chunk_size=500, chunk_overlap=50)

# MarkdownTextSplitter – splits on headings
from langchain.text_splitter import MarkdownTextSplitter

# SpacyTextSplitter – splits on sentences (requires spacy)
from langchain.text_splitter import SpacyTextSplitter
```

Chapter 9: Embeddings

9.1 What Are Embeddings?

An embedding is a dense vector (list of floating point numbers) that represents the semantic meaning of a piece of text. Similar meanings → similar vectors → similar positions in vector space. This is the foundation of semantic search.

Why vectors? Computers can't directly compare 'meaning'. But they can compute the distance between two points in N-dimensional space. We train neural networks to place semantically similar texts near each other in this space.

```
# Conceptual example (not real values)
embed("cat")      → [0.82, -0.31, 0.44, ...] # 768 dims
embed("kitten")   → [0.81, -0.29, 0.46, ...] # very close!
embed("automobile") → [0.12, 0.67, -0.55, ...] # far away
embed("car")      → [0.13, 0.65, -0.53, ...] # close to automobile
```

■ **Tip:** Classic word2vec test: $\text{embed}(\text{'king'}) - \text{embed}(\text{'man'}) + \text{embed}(\text{'woman'}) \approx \text{embed}(\text{'queen'})$. This shows embeddings capture analogy relationships as vector arithmetic.

9.2 Similarity Measures

Cosine Similarity: Measures the angle between two vectors. Range: -1 (opposite) to 1 (identical). Ignores vector magnitude — focuses purely on direction. Most common for text similarity because text length varies.

```
# Cosine similarity formula
# cos_sim(A, B) = (A · B) / (||A|| × ||B||)
# where A · B = sum of element-wise products
# and ||A|| = square root of sum of squares

import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Example
v1 = np.array([1.0, 0.5, -0.3])
v2 = np.array([0.9, 0.6, -0.2])
sim = cosine_similarity(v1, v2)
# sim ≈ 0.99 (very similar)
```

Euclidean Distance (L2): Straight-line distance between two points. Sensitive to vector magnitude. FAISS uses L2 by default. Lower = more similar.

Dot Product: Sum of element-wise products. Fast to compute. Used when vectors are normalised (same as cosine similarity for unit vectors).

9.3 all-mpnet-base-v2 Model

all-mpnet-base-v2 is a sentence-transformer model from HuggingFace. It is based on Microsoft's MPNet (Masked and Permuted Pre-Training), fine-tuned on 1 billion sentence pairs to produce high-quality sentence embeddings.

Property	Value
Base architecture	MPNet (12 layers, 768 hidden, 12 heads)
Output dimensions	768
Max input tokens	514 (text longer than this is truncated)
Training data	1B+ sentence pairs
Size on disk	~420 MB
Use case	Semantic similarity, sentence retrieval, clustering
Benchmark (SBERT)	State of the art as of 2021 release

```
from langchain_community.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2",
    model_kwargs={"device": "cpu"}, # or "cuda" for GPU
    encode_kwargs={"normalize_embeddings": True} # unit vectors for cosine sim
)

# Embed a single query
query_vector = embeddings.embed_query("What is machine learning?")
# Returns: List[float] of length 768

# Embed multiple documents (batched, more efficient)
doc_vectors = embeddings.embed_documents(["doc1 text", "doc2 text"])
# Returns: List[List[float]] shape: (2, 768)
```

9.4 Interview Questions — Embeddings

Q: Why use HuggingFace embeddings instead of OpenAI embeddings?

A: HuggingFace embeddings (all-mpnet-base-v2) run locally — no API calls, no cost, no rate limits, no data privacy concerns. OpenAI embeddings (text-embedding-ada-002, text-embedding-3-small) are higher quality but require API keys, incur cost (~\$0.0001/1K tokens), send your document to OpenAI's servers. For a demo project or privacy-sensitive data, local HuggingFace embeddings are preferred.

Q: What happens if a text chunk exceeds the embedding model's token limit?

A: all-mpnet-base-v2 has a 514 token limit. If a chunk exceeds this, the model silently truncates the input — only the first 514 tokens are embedded. The remaining text is ignored. This means large chunks may lose their endings. Solution: use RecursiveCharacterTextSplitter with a max chunk size of ~400 tokens (approximately 1600 characters) to stay safely within the limit.

Q: Explain cosine similarity intuitively.

A: Imagine two arrows pointing in 3D space from the origin. Cosine similarity measures how closely they point in the same direction, regardless of their length. Two arrows pointing in the exact same direction: similarity = 1.0 (identical meaning). Perpendicular arrows: similarity = 0 (unrelated). Opposite directions: similarity = -1 (opposite meanings). Length doesn't matter — a short summary and a long essay on the same topic both point in the same direction.

Chapter 10: FAISS — Vector Database

10.1 What is FAISS?

FAISS (Facebook AI Similarity Search) is a library for efficient similarity search in high-dimensional vector spaces. Developed by Meta AI Research and released in 2017. It is the most widely used vector search library for ML applications.

Problem it solves: given a database of 1 million 768-dimensional vectors and a query vector, find the top 10 most similar vectors. Brute force (compute distance to every vector) takes $O(N \times D)$ time — too slow for large N . FAISS provides data structures (indexes) that make this much faster.

10.2 Index Types

Index Type	Algorithm	Speed	Accuracy	Memory	Best For
IndexFlatL2	Exact brute force L2	Slowest	100%	High	Small DBs (<100K vectors), development
IndexFlatIP	Exact brute force inner product	Slowest	100%	High	Normalised vectors, cosine similarity
IndexIVFFlat	Inverted file + brute force	Fast	High	Medium	Medium DBs (100K-1M)
IndexIVFPQ	Inverted file + product quantisation	Fastest	Good	Low	Large DBs (1M+), disk-constrained
IndexHNSW	Hierarchical Navigable Small Worlds	Fast	Very high	Higher	General purpose ANN

10.3 What Happens in FAISS.from_texts()

```
# Under the hood of FAISS.from_texts(chunks, embeddings)
#
# Step 1: Embed all chunks
#   embedding_model.embed_documents(chunks)
#   → matrix of shape (N_chunks, 768)
#
# Step 2: Create a FAISS index
#   index = faiss.IndexFlatL2(768) # 768 = embedding dimension
#   (by default, LangChain uses IndexFlatL2)
#
# Step 3: Add vectors to index
#   index.add(np.array(all_vectors, dtype=np.float32))
#
# Step 4: Store the docstore (text↔index mapping)
#   docstore[doc_id] = Document(page_content=chunk, metadata={})
#
# Step 5: Return a FAISS VectorStore wrapping the index

# You can verify:
vectorstore = FAISS.from_texts(chunks, embeddings)
print(vectorstore.index.ntotal) # number of vectors stored
```

```
print(vectorstore.index.d)          # 768 – embedding dimension
```

10.4 What Happens in `similarity_search()`

```
# Under the hood of vectorstore.similarity_search(query, k=4)
#
# Step 1: Embed the query
#   query_vector = embedding_model.embed_query(query)
#   → shape: (768,)
#
# Step 2: FAISS search
#   distances, indices = index.search(
#       np.array([query_vector], dtype=np.float32), k=4
#   )
#   distances: array of L2 distances to top-4 matches
#   indices: array of database IDs of top-4 matches
#
# Step 3: Retrieve text for each index
#   results = [docstore[index_to_docid[i]] for i in indices[0]]
#
# Step 4: Return List[Document]
#   Each Document has .page_content and .metadata

docs = vectorstore.similarity_search("What is the revenue?", k=4)
for doc in docs:
    print(doc.page_content)
    print(doc.metadata) # {'source': 'report.pdf', 'page': 3}
```

10.5 Interview Questions — FAISS

Q: What is Approximate Nearest Neighbor (ANN) search and why use it?

A: ANN algorithms (like HNSW, IVF) find vectors that are approximately (not exactly) the nearest neighbors, much faster than exact search. For example, IndexIVFFlat partitions vectors into clusters (Voronoi cells) and only searches the nearest clusters — skipping most vectors. Trade-off: may miss the true nearest neighbor with small probability. For semantic search, near-nearest is semantically equivalent, so accuracy loss is acceptable and speed gain is enormous.

Q: How does FAISS store and retrieve vector-text mappings?

A: FAISS itself only stores and searches float vectors — it has no concept of text. LangChain's FAISS wrapper maintains two additional data structures: (1) `index_to_docstore_id`: maps FAISS integer IDs to document IDs, (2) `docstore`: an `InMemoryDocstore` mapping document IDs to `Document` objects. When `similarity_search` returns integer indices, LangChain uses these mappings to retrieve the original text.

Q: How would you persist the FAISS index to disk?

A: `vectorstore.save_local('faiss_index')` saves both the FAISS index file and the docstore pickle. Load with `FAISS.load_local('faiss_index', embeddings)`. This is important for production: you don't want to rebuild the

index (expensive) on every app restart.

Chapter 11: Large Language Models, Groq, and Llama

11.1 LLM Fundamentals

A Large Language Model is a neural network trained to predict the next token given a sequence of tokens. Trained on internet-scale text (trillions of tokens), LLMs learn grammar, facts, reasoning, and common sense as emergent properties of this prediction task.

Tokens: LLMs don't process characters or words — they process tokens. A token is roughly 3-4 characters or 0.75 words. 'unbelievable' might be 3 tokens: 'un', 'believ', 'able'. Tokenisation uses algorithms like BPE (Byte Pair Encoding) or WordPiece.

Context Window: the maximum number of tokens an LLM can process at once (input + output combined). Llama 3.3 70B: 128K tokens (~100K words). GPT-4o: 128K tokens. Claude 3: 200K tokens. Context window determines how much document text + question + instructions you can fit in one API call.

11.2 Transformer Architecture (Intuition)

Transformers use Self-Attention: for each token in a sequence, compute how much attention to pay to every other token. This captures long-range dependencies that earlier RNNs struggled with.

```
# Conceptual attention:
# "The bank that I visited was near the river."
#
# When encoding "bank" (token 3):
#   Attention to "river" (token 8): HIGH → financial bank or riverbank?
#   Attention to "visited" (token 5): HIGH → context clue
#   Attention to "The" (token 1): LOW → not informative
#
# This disambiguation happens in every layer, 32 layers for Llama 70B
# 70 BILLION parameters = massive capacity to learn patterns
```

11.3 Hallucinations

Hallucinations: LLMs generate plausible-sounding but factually incorrect text. Why? LLMs are trained to produce statistically likely continuations — they don't 'look up' facts, they recall patterns. If a fact wasn't in training data, or the question is ambiguous, the model confidently confabulates.

RAG solution: by grounding the LLM in retrieved context from your document, you reduce hallucinations. The prompt says 'answer ONLY from this context'. The model has less reason to confabulate when the answer is right there. But it's not perfect — LLMs can still ignore context or mix context with training data.

11.4 Groq

Groq is a company that built custom AI inference hardware: the Language Processing Unit (LPU). Their hardware is designed specifically for running LLM inference, unlike NVIDIA GPUs which are general-purpose.

Aspect	Groq LPU	NVIDIA GPU (A100)
Primary design	LLM inference (sequential token generation)	General matrix multiply (training + inference)
Inference speed	~750 tokens/second (Llama 70B)	~100-200 tokens/second
Memory bandwidth	Extremely high (low latency per token)	High but GPU overhead adds latency
Cost	Free tier available (rate limited)	~\$2-4/hour to rent
Latency (TTFT)	Very low (< 1 second)	Higher due to GPU loading overhead

11.5 Llama 3.3 70B

Llama (Large Language Model Meta AI) is Meta's open-source family of LLMs. Llama 3.3 70B is a 70 billion parameter instruction-tuned model. 'Instruction-tuned' means it was fine-tuned with RLHF (Reinforcement Learning from Human Feedback) to follow instructions, making it suitable for chat and Q&A; tasks.

```
from langchain_groq import ChatGroq

llm = ChatGroq(
    model="llama-3.3-70b-versatile", # model name on Groq
    temperature=0, # 0 = deterministic, 1 = creative
    max_tokens=1024, # max output tokens
    groq_api_key=os.getenv("GROQ_API_KEY")
)

# temperature controls randomness:
# 0.0: always picks most likely next token (deterministic)
# 0.7: some randomness (good for creative tasks)
# 1.0: high randomness (good for brainstorming)
# For Q&A;: use 0 or close to 0 for factual, grounded answers
```

■ **Tip:** Why not use OpenAI GPT-4? For a demo project: cost (GPT-4 is expensive), rate limits, and API key complexity. Groq offers a free tier with generous rate limits and Llama 3.3 70B is competitive with GPT-4 on many benchmarks.

Chapter 12: The Complete Question Answering Pipeline

12.1 Phase 1: Document Ingestion (One-Time)

This phase happens once when the user uploads a document and clicks 'Process'. It is computationally expensive (embedding generation) and should be cached.

12.1.1 File Upload & Type Detection

```
uploaded_file = st.file_uploader(...)
file_type = uploaded_file.name.split(".")[1].lower()
# "pdf", "docx", "jpg", "jpeg", "png"
```

12.1.2 Text Extraction (branch by type)

```
if file_type == "pdf":
    text = extract_pdf_text(uploaded_file)
    if len(text.strip()) < 100: # likely scanned
        text = ocr_pdf(uploaded_file)
elif file_type == "docx":
    text = docx2txt.process(uploaded_file)
elif file_type in ["jpg", "jpeg", "png"]:
    img = Image.open(uploaded_file)
    text = pytesseract.image_to_string(img)
```

12.1.3 Text Chunking

```
splitter = CharacterTextSplitter(
    separator="\n", chunk_size=1000, chunk_overlap=200
)
chunks = splitter.split_text(text)
# chunks: List[str], e.g., 45 chunks for a 10-page PDF
```

12.1.4 Embedding Generation

```
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2"
)
# Internally: calls embeddings.embed_documents(chunks)
# For 45 chunks of ~250 words each:
# Takes ~5-15 seconds on CPU, <1s on GPU
```

12.1.5 FAISS Index Construction

```
vectorstore = FAISS.from_texts(chunks, embeddings)
# Index built. Now in st.session_state:
```

```
st.session_state.vectorstore = vectorstore
st.success("Document processed! Ask your questions.")
```

12.2 Phase 2: Query Processing (Per Question)

This phase is fast (<2 seconds typically). It happens every time the user asks a question.

12.2.1 User Input Capture

```
user_question = st.text_input("Ask a question:")
if st.button("Ask") and user_question:
    # proceed with pipeline
```

12.2.2 Query Embedding

```
query_vector = embeddings.embed_query(user_question)
# Single vector, shape: (768,), takes <100ms on CPU
```

12.2.3 Similarity Search

```
relevant_docs = vectorstore.similarity_search(
    user_question, k=4
)
# Returns top 4 chunks as List[Document]
# Each Document: .page_content = chunk text
```

12.2.4 Prompt Construction

```
context = "\n\n".join([doc.page_content for doc in relevant_docs])
prompt = f"""Answer the question based ONLY on the context below.
If the answer isn't in the context, say "I don't know based on the document."

Context:
{context}

Question: {user_question}

Answer: ""
```

12.2.5 LLM Inference

```
response = llm.invoke(prompt)
answer = response.content # ChatGroq returns AIMessage
# Groq typically responds in < 1 second
```

12.2.6 Display and Store

```
st.write(answer)
st.session_state.chat_history.append({
```

```
        "role": "user", "content": user_question
    })
    st.session_state.chat_history.append({
        "role": "assistant", "content": answer
    })
```

12.3 End-to-End Timing Breakdown

Operation	Typical Time (CPU)	Notes
PDF text extraction	0.1 - 2s	Depends on page count and PDF complexity
OCR (per page)	3 - 10s	Much slower; preprocessing adds time
DOCX extraction	< 0.1s	XML parsing is fast
Text chunking	< 0.05s	Pure Python string operations
Embedding (45 chunks)	5 - 20s	Heaviest step; use GPU or cache
FAISS index build	< 0.1s	For < 10K chunks, IndexFlatL2 is instant
Query embedding	0.1 - 0.3s	Single vector
FAISS search	< 0.01s	Brute force over < 10K vectors is instant
Groq LLM inference	0.5 - 3s	Groq is exceptionally fast

Chapter 13: Environment Variables and Secrets Management

13.1 Why Environment Variables?

API keys are secrets: if someone gets your Groq API key, they can make API calls billed to your account. If you hardcode keys in source code and push to GitHub (even a private repo), they can be exposed through: accidental public repo, log files, error messages, repository cloning by collaborators.

Environment variables are set in the operating system environment — outside your code. Your code reads them at runtime. The key never appears in your source files.

```
# .env file (local development only)
# Add .env to .gitignore – NEVER commit this
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
HUGGINGFACE_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXX

# Python code reads it:
from dotenv import load_dotenv
import os

load_dotenv() # loads .env into os.environ

# Two ways to read:
key = os.getenv("GROQ_API_KEY")           # None if missing
key = os.environ.get("GROQ_API_KEY")       # None if missing
key = os.environ["GROQ_API_KEY"]           # KeyError if missing

# Best practice: validate at startup
if not os.getenv("GROQ_API_KEY"):
    raise EnvironmentError("GROQ_API_KEY not set. Check .env file.")
```

13.2 .gitignore for Secrets

```
# .gitignore
.env
.env.local
.env.production
venv/
__pycache__/
*.pyc
.streamlit/secrets.toml
faiss_index/
```

■ ■ If you accidentally commit a .env file with real keys: (1) Immediately revoke and regenerate the API key, (2) Remove it from git history using git filter-branch or BFG Repo Cleaner, (3) Never assume old commits are safe — bots scan GitHub in real time.

13.3 Streamlit Secrets Management

```
# .streamlit/secrets.toml (local development)
# Add to .gitignore!
GROQ_API_KEY = "gsk_XXXXXXXXXXXX"

# Reading in code:
api_key = st.secrets["GROQ_API_KEY"]
# or
api_key = st.secrets.get("GROQ_API_KEY")

# For Streamlit Cloud deployment:
# Go to: app settings → Secrets
# Paste your secrets in TOML format
# Streamlit injects them as st.secrets at runtime

# Fallback pattern (works locally and in cloud):
import os
api_key = st.secrets.get("GROQ_API_KEY") or os.getenv("GROQ_API_KEY")
```

Chapter 14: Deployment

14.1 requirements.txt

```
# Generate from active venv (includes all installed packages)
pip freeze > requirements.txt

# Result (example for DocQuest):
streamlit==1.35.0
langchain==0.2.5
langchain-community==0.2.5
langchain-groq==0.1.5
langchain-huggingface==0.0.3
faiss-cpu==1.8.0
sentence-transformers==3.0.0
PyPDF2==3.0.1
docx2txt==0.8
pytesseract==0.3.13
Pillow==10.3.0
python-dotenv==1.0.1
transformers==4.41.0
torch==2.3.0 # large! ~2GB download on Streamlit Cloud

# Tip: use exact versions (==) not ranges (>=) for reproducibility
# Tip: Streamlit Cloud has a 60-minute build timeout – large packages
# like torch can cause timeouts. Use CPU-only variants.
```

14.2 packages.txt (System Dependencies)

```
# packages.txt – system-level apt packages
# Required for Tesseract OCR on Streamlit Cloud (Ubuntu-based)
tesseract-ocr
tesseract-ocr-eng
poppler-utils # for pdf2image
libgl1        # for OpenCV (cv2)

# Streamlit Cloud runs: apt-get install -y $(cat packages.txt)
# at build time, before pip install -r requirements.txt
```

14.3 Streamlit Cloud Deployment Steps

- Push your code to a GitHub repository
- Go to share.streamlit.io and sign in with GitHub
- Click 'New app' → select your repo, branch, and app.py path
- Add secrets: App settings → Secrets → paste TOML
- Click Deploy — Streamlit clones repo, installs packages, starts app

- App URL: `https://[your-username]-[app-name]-[random].streamlit.app`

14.4 Common Deployment Errors

Error	Cause	Fix
ModuleNotFoundError: No module named <code>xxx</code>	Package missing from requirements.txt	Add to requirements.txt and redeploy
tesseract is not installed or not in PATH	Tesseract not in packages.txt	Add 'tesseract-ocr' to packages.txt
GROQ_API_KEY not found	Secret not added in Streamlit Cloud	Add in App Settings → Secrets
MemoryError or OOM	HuggingFace model too large for free tier	Use smaller model or upgrade plan
Build timeout (60 min)	torch or transformers too large to install	Use pre-built Docker image or lighter model
FAISS ImportError	Used faiss-gpu but no GPU in cloud	Change to faiss-cpu in requirements.txt

Chapter 15: 150+ Technical Interview Questions

— PYTHON FUNDAMENTALS —

Q: What is the difference between a list and a tuple in Python?

A: Lists are mutable (can be changed after creation) and use square brackets []. Tuples are immutable (cannot be changed) and use parentheses (). Tuples are faster for iteration and can be used as dictionary keys. Use tuples for fixed data (coordinates, RGB values, config constants) and lists for collections that change.

Q: Explain Python's GIL (Global Interpreter Lock).

A: The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. This means Python threads cannot achieve true parallelism for CPU-bound tasks. Solution: use multiprocessing (separate processes, each with their own GIL) for CPU work, or use asyncio/threading for I/O-bound work. For ML inference, use libraries like NumPy/PyTorch that release the GIL during computation.

Q: What is a generator in Python and when would you use one?

A: A generator is a function that uses 'yield' instead of 'return' to produce a sequence of values lazily (one at a time). Unlike lists, generators don't hold all values in memory. In DocQuest, if processing thousands of documents, a generator that yields chunks one at a time saves memory compared to returning a list of all chunks.

Q: Explain decorators in Python with an example.

A: @decorator syntax wraps a function with another function, adding behaviour before/after. Example: @st.cache_data wraps your function so Streamlit intercepts calls, checks a cache, and returns the cached result instead of calling the original function. Implemented as functions that take a function and return a new function.

Q: What is *args and **kwargs?

A: *args collects positional arguments into a tuple. **kwargs collects keyword arguments into a dict. Used to write functions that accept variable numbers of arguments. LangChain uses **kwargs extensively for passing configuration options through the chain.

Q: What is a context manager and how does 'with' work?

A: A context manager manages resources: it guarantees setup (__enter__) and cleanup (__exit__) happen, even if an exception occurs. 'with open("file") as f:' calls f.__enter__() on entry and f.__exit__() on exit. Useful for files (auto-close), database connections, locks, timers.

Q: Difference between deepcopy and shallow copy?

A: Shallow copy: copies the outer object, but nested objects are shared references. `deepcopy`: recursively copies all nested objects. In DocQuest, if you copy `session_state` shallowly, modifying a nested list modifies the original too. Use `copy.deepcopy()` when you need true independence.

Q: What is list comprehension and dict comprehension?

A: List comprehension: `[expression for item in iterable if condition]`. More Pythonic and often faster than equivalent for loop. Dict comprehension: `{key: value for item in iterable}`. Example in DocQuest: `chunk_lengths = {i: len(c) for i, c in enumerate(chunks)}`.

— STREAMLIT —

Q: Why does Streamlit rerun the script on every interaction?

A: Streamlit's programming model is reactive: the UI is a function of state. Any state change (widget interaction) triggers a re-execution to recompute the UI. This simplifies programming (no callbacks) but requires caching for expensive operations.

Q: What is the difference between `st.cache_data` and `st.cache_resource`?

A: `@st.cache_data` caches data objects; creates a copy per caller (thread-safe). `@st.cache_resource` caches shared objects like ML models; returns the SAME instance to all callers (faster, but not thread-safe for mutation). Use `@st.cache_resource` for HuggingFace models.

Q: How do you handle file uploads in Streamlit?

A: `st.file_uploader()` returns an `UploadedFile` object (file-like) or `None`. Check if it's not `None` before processing. Important: after calling `.read()`, the file pointer is at the end — call `.seek(0)` to reset before reading again.

Q: How do you implement a multi-turn chat in Streamlit?

A: Store conversation history in `st.session_state.messages` (list of dicts with 'role' and 'content'). On each rerun, iterate messages to render history, then append new user message and assistant response to the list.

Q: What is Streamlit's execution model vs Flask?

A: Flask: event-driven — code runs only when an HTTP endpoint is called. Streamlit: re-executes the ENTIRE script on every interaction. Streamlit is simpler but less efficient for complex state management.

— PDF & DOCUMENT PROCESSING —

Q: Why does PyPDF2 sometimes return garbled text?

A: PDFs store text as glyph IDs mapped through font encoding tables (ToUnicode CMaps). If the CMap is missing, corrupted, or uses custom encoding, PyPDF2 cannot decode glyphs to Unicode characters. Try `pdfplumber` (more robust extraction) or OCR as fallback.

Q: What is the difference between a searchable and scanned PDF?

A: Searchable PDFs have a text layer — characters are stored as Unicode. Scanned PDFs are images in PDF containers — no text layer. PyPDF2 returns empty or garbage for scanned PDFs; OCR is required.

Q: How does DOCX differ from DOC format?

A: DOCX is an Open XML format — a ZIP archive containing XML files. Open, standardised, easily processed without Word. DOC is a proprietary binary format (.doc) — requires specific libraries (win32com on Windows) and is harder to process. Always prefer DOCX for programmatic processing.

— LANGCHAIN —

Q: What is LangChain Expression Language (LCEL)?

A: LCEL is LangChain's modern composition system using the `|` pipe operator. Components (prompts, LLMs, parsers) implement the Runnable interface with `.invoke()`, `.stream()`, `.batch()`, and `.ainvoke()` methods. LCEL enables built-in streaming, async, parallel execution, and better debugging compared to old chain classes.

Q: What is the difference between RetrievalQA and a custom LCEL chain?

A: RetrievalQA is a high-level convenience class with sensible defaults. A custom LCEL chain gives full control: you can modify the retrieval step, add re-ranking, inject metadata, add fallback logic, stream responses. For production, custom LCEL chains are preferred.

Q: What is LangChain Memory and how does it work?

A: Memory persists conversation history across LLM calls. ConversationBufferMemory stores all messages. On each call, it prepends the history to the prompt. ConversationSummaryMemory uses an LLM to compress history into a summary, useful for long conversations that would exceed context window.

Q: Explain the 'stuff' chain type in RetrievalQA.

A: 'stuff' takes all retrieved documents, concatenates them into a single context string, and stuffs it into one prompt. Simple and works for small documents. For larger contexts, use 'map_reduce' (process chunks separately and combine) or 'refine' (iteratively improve the answer with each chunk).

— EMBEDDINGS & VECTOR SEARCH —

Q: What is cosine similarity and why is it used for embeddings?

A: Cosine similarity measures the angle between two vectors, ranging from -1 to 1. It ignores magnitude (document length) and focuses on direction (semantic content). Two short and long texts about the same topic have similar directions but different magnitudes — cosine similarity correctly identifies them as similar.

Q: What is the all-mpnet-base-v2 model?

A: A sentence-transformer model based on Microsoft's MPNet, fine-tuned on 1B+ sentence pairs for semantic similarity. Outputs 768-dimensional embeddings. Supports up to 514 tokens input. State-of-the-art performance on semantic textual similarity benchmarks (STS). Available free on HuggingFace.

Q: How does FAISS similarity search work at a high level?

A: FAISS embeds the query into a vector, then searches an index for the k nearest vectors by L2 distance or inner product. For IndexFlatL2 (used by default in LangChain): brute-force comparison against all stored vectors. Fast for <100K vectors. For larger datasets, use IVFFlat or HNSW (approximate search).

Q: What is the difference between semantic search and keyword search?

A: Keyword search (BM25, TF-IDF) matches documents based on exact word overlap. A query for 'automobile' won't find documents about 'car'. Semantic search uses embeddings to find documents with similar meaning regardless of exact words. 'automobile' and 'car' are close in embedding space. DocQuest uses semantic search.

Q: How would you handle a query in a language different from the document?

A: Use a multilingual embedding model like paraphrase-multilingual-mpnet-base-v2 (supports 50+ languages). It maps text from different languages into the same vector space, so a French query finds relevant English chunks. all-mpnet-base-v2 is English-only.

— LARGE LANGUAGE MODELS —

Q: What is a token in the context of LLMs?

A: A token is the basic unit of text an LLM processes. Using Byte Pair Encoding (BPE), common words are single tokens, rare words are split into subword tokens. 'unbelievable' might be ['un', 'believ', 'able'] = 3 tokens. 1 token $\approx$ 4 characters $\approx$ 0.75 words. Context window limits are in tokens.

Q: What is temperature in LLM inference?

A: Temperature controls the randomness of token selection. At temperature=0, the model always picks the highest-probability next token (deterministic). At temperature=1, probability distribution is unchanged (moderate randomness). Above 1, distribution is flattened (more random). For Q&A; (factual tasks), use temperature=0 or 0.1.

Q: What causes LLM hallucinations?

A: LLMs are trained to generate statistically likely text, not factually correct text. They don't 'know' facts — they recall patterns. Hallucinations occur when: (1) the fact wasn't in training data, (2) the question is ambiguous, (3) the model is asked to speculate, (4) conflicting training data creates uncertainty. RAG reduces hallucinations by providing direct source material.

Q: Why is Groq faster than traditional GPU inference?

A: Groq's LPU (Language Processing Unit) is custom silicon designed specifically for sequential token generation. Its architecture eliminates the memory bandwidth bottlenecks that GPU inference suffers from.

The LPU has a deterministic execution model with no cache misses, enabling consistent sub-second first-token latency.

Q: What is the difference between GPT-4 and Llama 3.3 70B?

A: GPT-4: proprietary (OpenAI), closed weights, API-only, expensive, very high quality. Llama 3.3 70B: open-source (Meta), downloadable weights, can run locally, free (as weights), competitive quality. For production: GPT-4 leads on complex reasoning. For privacy, cost, and local deployment: Llama is preferred.

Q: What is context window and why does it matter for RAG?

A: Context window is the max tokens the LLM processes at once. Matters for RAG because you fit: system prompt + retrieved chunks + conversation history + user question + expected answer all within this limit. Llama 3.3 70B: 128K tokens. Practical RAG budget: leave ~50% for context, ~25% for conversation, ~25% for output.

— SYSTEM DESIGN & ARCHITECTURE —

Q: How would you scale DocQuest for 1000 simultaneous users?

A: 1) Separate ingestion from query service. 2) Pre-compute and persist FAISS indexes. 3) Use a shared vector database (Pinecone/Weaviate) instead of per-session FAISS. 4) Deploy behind a load balancer with multiple Streamlit/FastAPI instances. 5) Cache embedding model with @cache_resource. 6) Use async LLM calls. 7) Add Redis for session state. 8) Queue heavy processing (embedding) with Celery.

Q: How would you add authentication to DocQuest?

A: Options: (1) Streamlit Authenticator library — simple username/password in secrets.toml. (2) OAuth2 (Google/GitHub) via streamlit_oauth. (3) Move to FastAPI backend + React frontend for production-grade auth with JWT tokens. (4) Cloudflare Access (zero-trust, enterprise). For a student project: streamlit-authenticator is sufficient.

Q: What database would you use to persist documents and indexes?

A: Documents and extracted text: PostgreSQL or MongoDB. FAISS indexes: save to disk with vectorstore.save_local() and load on demand. Production-grade vector storage: Pinecone (managed, scalable), Weaviate (open-source, hybrid search), Qdrant (open-source, fast). Metadata (user, upload date, file name): PostgreSQL.

Q: How would you add support for multi-document Q&A;?

A: 1) Allow multiple file uploads. 2) Process each document into its own list of chunks. 3) Merge all chunks into a single FAISS index. 4) Add metadata (source filename) to each chunk's Document.metadata. 5) In answers, cite the source document. 6) Optionally: let users filter by document.

— RAG & GENAI ENGINEERING —

Q: What are the main failure modes of RAG?

A: (1) Retrieval failure: relevant chunks not retrieved (improve chunking, use MMR). (2) Context hallucination: LLM ignores retrieved context (strengthen prompt: 'ONLY answer from context'). (3) Chunk boundary issues: answer spans two chunks (increase overlap). (4) Irrelevant retrieval: wrong chunks returned (improve embedding model or add re-ranking). (5) Context window overflow: too many chunks sent (reduce k or chunk size).

Q: What is MMR (Maximal Marginal Relevance) retrieval?

A: Standard similarity search returns k most similar chunks — but they may all cover the same information (redundant). MMR balances relevance with diversity: selected chunks are similar to the query AND dissimilar to already-selected chunks. Result: more diverse, comprehensive context. Use: `vectorstore.as_retriever(search_type='mmr')`.

Q: What is re-ranking and how does it improve RAG?

A: First-stage retrieval (FAISS) uses fast approximate search to get top-50 candidates. Re-ranking uses a more precise cross-encoder model (e.g., ms-marco-MiniLM) to score query-chunk pairs more accurately and reorder. The top-5 re-ranked results are sent to the LLM. This improves precision at the cost of latency. Libraries: sentence-transformers, Cohere Rerank API.

Q: What is HyDE (Hypothetical Document Embeddings)?

A: Trick to improve retrieval: instead of embedding the user's question directly, prompt the LLM to generate a hypothetical answer, then embed THAT. The hypothetical answer is stylistically closer to the actual document chunks than the question is. Embed the hypothetical answer and use it for FAISS search. Often improves recall.

Q: How would you evaluate a RAG pipeline?

A: Metrics: (1) Faithfulness: does the answer contain only information from context? (use LLM-as-judge). (2) Answer Relevance: does the answer address the question? (embedding similarity). (3) Context Precision: are retrieved chunks actually relevant? (4) Context Recall: are all relevant chunks retrieved? Frameworks: RAGAS, TruLens, DeepEval.

— DEPLOYMENT & DEVOPS —

Q: What should be in .gitignore for a Python ML project?

A: `.env` (secrets), `venv/` (virtual environment), `__pycache__/` and `*.pyc` (bytecode), `.streamlit/secrets.toml` (secrets), `*.pkl` (large model files), `faiss_index/` (FAISS index), `data/` (potentially large datasets), `*.h5` and `*.pt` (model weights).

Q: What is the difference between faiss-cpu and faiss-gpu?

A: `faiss-cpu` is a CPU-only build of FAISS — works everywhere, no GPU required. `faiss-gpu` builds use CUDA for GPU acceleration — 10-100x faster for large indexes and batch searches. For DocQuest (small documents, simple queries), `faiss-cpu` is sufficient. For production with millions of vectors, use `faiss-gpu`.

with an A100/H100.

Q: How do you handle large model downloads on Streamlit Cloud?

A: HuggingFace models download on first use and are cached in `~/.cache/huggingface/hub`. On Streamlit Cloud, this cache persists between sessions but resets on restart. For reproducibility: pin the model version in your code. For faster startup: use a pre-built Docker image with the model pre-downloaded. For smaller deployments: use smaller models (all-MiniLM-L6-v2, 80MB vs all-mpnet-base-v2, 420MB).

Chapter 16: Project Viva Questions

The following questions simulate a real viva examination by your interviewer. For each, the ideal answer, common mistakes, and follow-up questions are provided.

Walk me through your project in 2 minutes.

■ Ideal Answer:

DocQuest is an AI-powered document Q&A system. Users upload PDFs, DOCX files, or images. The app extracts text (PyPDF2 for PDFs, docx2txt for DOCX, pytesseract for images). I split the text into overlapping chunks using LangChain's CharacterTextSplitter. Each chunk is converted to a 768-dimensional embedding using HuggingFace's all-mpnet-base-v2 model. These embeddings are stored in a FAISS vector index. When a user asks a question, the question is embedded using the same model, FAISS finds the top 4 most similar chunks, and a prompt with those chunks + the question is sent to Groq's Llama 3.3 70B model, which returns the answer. The UI is built with Streamlit.

■ Common Wrong Answer:

Vague description: 'I built an app that answers questions from documents using AI.' No mention of specific technologies, the RAG pipeline, or how the components connect.

■ Follow-Up Questions:

- Why did you choose FAISS over ChromaDB?
- How does your app handle a 200-page PDF?
- What happens if the answer isn't in the document?

Why did you use CharacterTextSplitter with chunk_size=1000 and chunk_overlap=200? Justify these numbers.

■ Ideal Answer:

1000 characters is approximately 200-250 words or 150-180 tokens — well within all-mpnet-base-v2's 514-token limit and small enough that each chunk is semantically focused. 200 characters overlap (~40 words) prevents answer fragmentation at chunk boundaries. These are widely-used defaults in the LangChain community for general documents. I could tune them: smaller chunks (500 chars) for precise retrieval, larger chunks (2000 chars) for complex reasoning questions. I chose these balanced defaults after testing on sample documents.

■ Common Wrong Answer:

These are the default values. / I just used whatever values I found online.

■ Follow-Up Questions:

- How would you evaluate whether different chunk sizes improve answer quality?
- What chunk size would you use for a legal contract vs a news article?

- What is RecursiveCharacterTextSplitter and when is it better?
-

Explain exactly what FAISS.from_texts() does under the hood.

■ Ideal Answer:

It: (1) calls `embedding_model.embed_documents(chunks)` to get a matrix of vectors, shape (N, 768). (2) creates a `faiss.IndexFlatL2(768)` index (exact brute-force L2 search). (3) calls `index.add(vectors)` to store all vectors. (4) creates a docstore (`InMemoryDocstore`) mapping document IDs to Document objects. (5) creates an `index_to_docstore_id` mapping from integer FAISS IDs to docstore keys. (6) returns a LangChain FAISS object wrapping all of this. The returned object supports `similarity_search()`, `save_local()`, `load_local()`, and `as_retriever()`.

■ Common Wrong Answer:

It takes the chunks and creates an embedding from them.

■ Follow-Up Questions:

- How would you save and reload the FAISS index?
 - What index would you use for 10 million vectors?
 - What is the difference between `IndexFlatL2` and `IndexIVFFlat`?
-

Your app processes documents client-side or server-side? Explain the implications.

■ Ideal Answer:

All processing is server-side (on the Streamlit server). The user's browser uploads the file to the Streamlit server via HTTP. The server runs PyPDF2, embeds chunks, builds the FAISS index, and calls the Groq API. This has implications: (1) privacy — the document is sent to and processed on the server (and to Groq's API); for sensitive documents this is a concern, (2) scalability — each user gets their own in-memory FAISS index; with many users, memory consumption grows, (3) performance — depends on server resources, not user hardware.

■ Common Wrong Answer:

Client-side, because the user uploads from their computer.

■ Follow-Up Questions:

- How would you make DocQuest run entirely in the browser (client-side)?
 - What are the privacy implications of sending documents to Groq?
 - How would you add document encryption?
-

How does your app handle a user who uploads a scanned PDF?

■ Ideal Answer:

The app tries PyPDF2 text extraction first. If the returned text is empty or has fewer than N characters (threshold check), the app falls back to OCR: it uses `pdf2image` to convert PDF pages to PIL Images, then

applies `pytesseract.image_to_string()` to each image. The OCR text may be noisier than direct extraction (recognition errors, spacing issues), but it enables Q&A on scanned documents. I could improve this by: preprocessing images (grayscale, denoise, deskew) before OCR, and post-processing OCR output to fix common errors.

■ **Common Wrong Answer:**

It returns an error / It doesn't work on scanned PDFs.

■ **Follow-Up Questions:**

- What image preprocessing steps would you apply before OCR?
 - What is Tesseract's confidence score and how would you use it?
 - What alternative OCR tools exist?
-

What happens if the user asks a question that is NOT answered anywhere in the document?

■ **Ideal Answer:**

FAISS will still return the top 4 chunks (by similarity) regardless of whether they contain the answer — FAISS always returns k results. The prompt instructs the LLM: 'If the answer is not in the context, say I don't know based on the provided document.' A well-instructed LLM should acknowledge uncertainty. However, LLMs sometimes ignore this instruction and hallucinate an answer. To improve reliability: (1) strengthen the instruction, (2) check if similarity scores are below a threshold (low score = irrelevant chunks), (3) add a verification step asking the LLM if the context contains the answer before generating it.

■ **Common Wrong Answer:**

It returns an error. / It will not answer.

■ **Follow-Up Questions:**

- How would you implement a similarity threshold to detect off-topic queries?
 - What is the confidence score in FAISS and how do you access it?
 - How would you tell the user which document section the answer came from?
-

What is RAG and why is it better than fine-tuning for this use case?

■ **Ideal Answer:**

RAG (Retrieval-Augmented Generation) retrieves relevant context from a knowledge base at inference time and provides it to the LLM. Fine-tuning trains the LLM on your data to bake knowledge into its weights. For DocQuest's use case (user uploads arbitrary documents at runtime): fine-tuning is impractical — you cannot fine-tune the model for each user's document. RAG is ideal because it works with any document at inference time without any training. Fine-tuning is better when: you have a large, fixed dataset and want the model to learn new skills or domain-specific reasoning patterns.

■ **Common Wrong Answer:**

RAG is better because it's newer than fine-tuning.

■ Follow-Up Questions:

- What is 'full fine-tuning' vs 'LoRA fine-tuning'?
 - How would you combine RAG and fine-tuning?
 - What are the limitations of RAG?
-

How does Streamlit's `session_state` work and why do you need it?

■ Ideal Answer:

Streamlit re-executes the entire script on every user interaction. Without `session_state`, variables would be reset on every rerun — the FAISS vectorstore built during document processing would be lost when the user types a question. `session_state` is a server-side dictionary keyed by session ID (from the user's browser). It persists Python objects across reruns within the same session. I store the vectorstore, chat history, and processing status in `session_state` so they survive reruns.

■ Common Wrong Answer:

It's a dictionary for storing variables.

■ Follow-Up Questions:

- What is the difference between `session_state` and a global variable in Streamlit?
 - How does `session_state` scale with multiple users?
 - What types of objects can be stored in `session_state`?
-

Chapter 17: Code Walkthrough

17.1 Project File Structure

```
DocQuest/
├── app.py                # Main Streamlit application
├── requirements.txt       # Python dependencies
├── packages.txt          # System dependencies (for Streamlit Cloud)
├── .env                  # Local secrets (NEVER commit)
├── .gitignore            # Files excluded from git
├── .streamlit/
│   └── secrets.toml      # Local Streamlit secrets (NEVER commit)
├── utils/                # Optional utility modules
│   ├── pdf_processor.py  # PDF extraction logic
│   ├── ocr_processor.py  # OCR logic
│   └── qa_pipeline.py    # Q&A; chain logic
```

17.2 app.py Line-by-Line Walkthrough

Imports block

```
import os
import streamlit as st
from dotenv import load_dotenv
from PyPDF2 import PdfReader
import docx2txt
from PIL import Image
import pytesseract
from langchain.text_splitter import CharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_groq import ChatGroq
from langchain.chains import RetrievalQA
from langchain_core.prompts import PromptTemplate
```

Each import serves a purpose. 'os' for environment variables. 'streamlit as st' for the UI. 'dotenv' to load .env. 'PdfReader' to extract text from PDFs. 'docx2txt' for Word documents. 'Image' and 'pytesseract' for OCR. LangChain components for the RAG pipeline.

Environment setup

```
load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY") or st.secrets.get("GROQ_API_KEY")
```

load_dotenv() reads the .env file and populates os.environ. The fallback to st.secrets makes the app work both locally (using .env) and on Streamlit Cloud (using dashboard secrets).

Page configuration

```
st.set_page_config(
    page_title="DocQuest - AI Document Q&A;",
    page_icon="📄",
    layout="wide"
)
```

Must be the first Streamlit call. layout='wide' uses the full browser width. Emojis in page_icon are supported.

Session state initialisation

```
if "vectorstore" not in st.session_state:
    st.session_state.vectorstore = None
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []
if "doc_processed" not in st.session_state:
    st.session_state.doc_processed = False
```

Defensive initialisation: only set defaults on FIRST run (check 'not in'). This pattern prevents resetting state on reruns triggered by user interactions.

Text extraction function

```
def extract_text(uploaded_file) -> str:
    file_name = uploaded_file.name.lower()
    if file_name.endswith(".pdf"):
        return extract_from_pdf(uploaded_file)
    elif file_name.endswith(".docx"):
        return docx2txt.process(uploaded_file)
    elif file_name.endswith((".jpg", ".jpeg", ".png")):
        img = Image.open(uploaded_file)
        return pytesseract.image_to_string(img)
    return ""
```

Dispatcher function: routes to correct extractor based on file extension. Returns empty string as safe default for unknown types. Using .lower() prevents case-sensitivity bugs ('PDF' vs 'pdf').

PDF extraction with OCR fallback

```
def extract_from_pdf(pdf_file) -> str:
    reader = PdfReader(pdf_file)
    text = ""
    for page in reader.pages:
        text += page.extract_text() or ""
    if len(text.strip()) < 50: # likely scanned
        pdf_file.seek(0) # reset file pointer
        from pdf2image import convert_from_bytes
        images = convert_from_bytes(pdf_file.read())
        text = " ".join([pytesseract.image_to_string(img) for img in images])
    return text
```

Two-stage extraction: direct text extraction first (fast), OCR fallback for scanned PDFs. The threshold of 50 characters detects near-empty extractions. `seek(0)` is critical: after `PdfReader` reads the file, the pointer is at EOF — reset before re-reading for `pdf2image`.

Chunking and vector store creation

```
@st.cache_data
def process_document(text: str):
    splitter = CharacterTextSplitter(
        separator="\n", chunk_size=1000, chunk_overlap=200
    )
    chunks = splitter.split_text(text)
    return chunks

@st.cache_resource
def get_embeddings():
    return HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-mpnet-base-v2"
    )
```

`@st.cache_data` on `process_document`: same text → same chunks (no recomputation). `@st.cache_resource` on `get_embeddings`: expensive model load (~5s) done only once, shared across all requests.

17.3 Possible Code Improvements

- Add document metadata tracking: store filename, upload date, page count in `session_state`
- Implement similarity score threshold: if max similarity < 0.3, answer 'No relevant information found'
- Add source citation: display which chunk (and page number) the answer came from
- Use `RecursiveCharacterTextSplitter` instead of `CharacterTextSplitter` for better boundary handling
- Add async processing: use `asyncio` + `st.spinner` to show progress during embedding
- Persist FAISS index: save to disk so users can reload previous sessions
- Add multi-document support: merge indexes from multiple uploads
- Implement conversation memory: include previous Q&A; in the prompt context
- Add download button: let users export the Q&A; session as PDF
- Add input validation: reject files over size limit, unsupported formats

Chapter 18: Optimization and Production Architecture

18.1 Advanced RAG Techniques

```
# Hybrid Search: combine semantic + keyword search
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

bm25_retriever = BM25Retriever.from_texts(chunks, k=4)
faiss_retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

# Ensemble: weighted combination of results
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, faiss_retriever],
    weights=[0.5, 0.5] # equal weight to both
)

# Cross-encoder re-ranking
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
query = "What is the revenue for Q3?"
candidates = hybrid_retriever.invoke(query)
pairs = [(query, doc.page_content) for doc in candidates]
scores = reranker.predict(pairs)
ranked_docs = [doc for _, doc in sorted(zip(scores, candidates), reverse=True)]
top_3 = ranked_docs[:3]
```

18.2 Production Database: ChromaDB and Pinecone

```
# ChromaDB – persistent, local, free
from langchain_community.vectorstores import Chroma

vectorstore = Chroma.from_texts(
    chunks,
    embeddings,
    persist_directory="./chroma_db" # persists to disk
)
# No need for save_local/load_local – persists automatically

# Pinecone – managed cloud, scalable, production
import pinecone
from langchain_pinecone import PineconeVectorStore

pc = pinecone.Pinecone(api_key=os.getenv("PINECONE_API_KEY"))
index = pc.Index("docquest-index")
```

```
vectorstore = PineconeVectorStore(
    index=index,
    embedding=embeddings,
    namespace="user_123" # isolate per user
)
```

18.3 LangGraph for Complex Workflows

```
# LangGraph: stateful, graph-based LLM workflows
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class AgentState(TypedDict):
    question: str
    context: List[str]
    answer: str
    needs_clarification: bool

def retrieve_node(state: AgentState) -> AgentState:
    docs = retriever.invoke(state["question"])
    state["context"] = [doc.page_content for doc in docs]
    return state

def generate_node(state: AgentState) -> AgentState:
    context = "\n".join(state["context"])
    response = llm.invoke(f"Context: {context}\nQ: {state['question']}")
    state["answer"] = response.content
    return state

def check_quality(state: AgentState) -> str:
    if len(state["answer"]) < 20:
        return "needs_clarification"
    return "done"

# Build graph
graph = StateGraph(AgentState)
graph.add_node("retrieve", retrieve_node)
graph.add_node("generate", generate_node)
graph.add_edge("retrieve", "generate")
graph.add_conditional_edges("generate", check_quality,
    {"done": END, "needs_clarification": "retrieve"})
graph.set_entry_point("retrieve")
app_graph = graph.compile()
```

18.4 Production Architecture with Docker

```
# Dockerfile for DocQuest
FROM python:3.11-slim
```

```

# Install system dependencies
RUN apt-get update && apt-get install -y \
    tesseract-ocr tesseract-ocr-eng \
    poppler-utils libgl1 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy and install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Pre-download HuggingFace model (avoids runtime download)
RUN python -c "from sentence_transformers import SentenceTransformer; \
    SentenceTransformer('sentence-transformers/all-mpnet-base-v2')"

# Copy application code
COPY . .

EXPOSE 8501

CMD ["streamlit", "run", "app.py", "--server.port=8501",
    "--server.address=0.0.0.0"]

# docker-compose.yml for local development
version: '3.8'
services:
  docquest:
    build: .
    ports:
      - "8501:8501"
    environment:
      - GROQ_API_KEY=${GROQ_API_KEY}
    volumes:
      - ./data:/app/data
  redis:
    image: redis:alpine
    ports:
      - "6379:6379"

```

18.5 Async Processing with FastAPI Backend

```

# Move heavy processing to FastAPI for better scalability
# fastapi_backend.py

from fastapi import FastAPI, UploadFile, BackgroundTasks
from fastapi.responses import JSONResponse
import asyncio

app = FastAPI()

```

```

processing_jobs = {} # In production: use Redis

@app.post("/upload")
async def upload_document(
    file: UploadFile,
    background_tasks: BackgroundTasks
):
    job_id = str(uuid.uuid4())
    background_tasks.add_task(process_document_async, job_id, file)
    return {"job_id": job_id, "status": "processing"}

@app.get("/status/{job_id}")
async def check_status(job_id: str):
    return processing_jobs.get(job_id, {"status": "not_found"})

@app.post("/ask")
async def ask_question(query: QueryRequest):
    vectorstore = load_vectorstore(query.document_id)
    answer = await get_answer_async(query.question, vectorstore)
    return {"answer": answer}

```

18.6 Improvement Roadmap Summary

Priority	Improvement	Impact	Effort
High	Persist FAISS index to disk/DB	Users keep history across sessions	Low
High	Add similarity threshold	Reduces hallucination on off-topic queries	Low
High	Source citation in answers	More trustworthy, verifiable answers	Medium
High	RecursiveCharacterTextSplitter	Better chunk boundaries	Low
Medium	Hybrid search (BM25 + FAISS)	Better retrieval accuracy	Medium
Medium	Cross-encoder re-ranking	Top-N results are more precise	Medium
Medium	Move to ChromaDB	Persistent storage, no reprocessing	Medium
Medium	Conversation memory	Multi-turn follow-up questions work better	Medium
Low	Docker deployment	Reproducible, scalable production deployment	High
Low	Pinecone integration	Scales to millions of documents	High
Low	Authentication	Multi-user with isolated document stores	High
Low	LangGraph workflow	Complex multi-step reasoning	High

Final Note

You have now studied the complete technical foundation of DocQuest. Every component — from raw PDF bytes to a natural language answer — has been explained with depth, intuition, and interview-oriented framing.

Master the RAG pipeline, understand why each component exists, and be able to discuss tradeoffs and improvements confidently. Good luck in your interviews!